UNIVERSIDAD TECNOLÓGICA DE GUTIÉRREZ ZAMORA

PROGRAMA EDUCATIVO:

Ingeniería en Desarrollo y Gestión de Software

NOMBRE DE LA MATERIA:

Desarrollo Móvil Integral

NOMBRE DE LA TAREA:

Unidad II — Seguridad MASVS

Auditoría básica de seguridad con OWASP MASVS / MASTG
(aplicación propia: TotoNinin)

ELABORADO POR:

Héctor Zahid Juárez Simbrón

DOCENTE:

Lothar Yamir Hernández Gaona

1010 - “AEI”

SEPTIEMBRE – DICIEMBRE 2026

Índice

Alcance y controles seleccionados....3
Procedimiento de prueba....6
Resultados y evidencias....8
Hallazgos y nivel de riesgo....12
Correcciones implementadas....19
Repetición de pruebas y conclusiones....21
Entrega y repositorio....24

Alcance y controles seleccionados

Objeto de la auditoría

Aplicación auditada: TotoNinin — sistema de gestión de eventos (registro de usuarios, calendario/categorías de eventos, comentarios y panel administrativo).

Dato	Valor
Tecnología	PHP 8 + MySQL/MariaDB, renderizado del lado del servidor (sin framework; mysqli procedural y orientado a objetos mezclados)
Plataforma	Aplicación web. No se incluyó en el ZIP entregado ningún empaquetado móvil nativo (APK/IPA) ni proyecto híbrido (Ionic/Cordova/Capacitor) para TotoNinin — ver la aclaración de alcance en «Aclaración de alcance: MASVS aplicado a una aplicación web».
Versión / snapshot auditado	v1-original: snapshot del archivo .zip entregado el 29/10/2026 (sin historial de Git previo). v2-corregida: mismo código con las 6 correcciones de esta auditoría aplicadas. Ambas versiones se incluyen íntegras en el repositorio entregado (ver «Entrega y repositorio»).
Entorno de prueba	Contenedor Linux aislado, PHP 8.3.6 (servidor embebido php -S). No fue posible instalar MariaDB/MySQL en el entorno de auditoría (sin acceso al repositorio de paquetes del sistema operativo), por lo que se usó una base de datos de prueba equivalente en SQLite con una capa de compatibilidad que traduce las mismas llamadas mysqli que usa la aplicación (pruebas/mysqli_shim.php), sin añadir ni quitar ninguna lógica de seguridad. Cada hallazgo se verificó además por revisión estática directa del código fuente PHP real.
Cuentas y datos de prueba	Usuarios ficticios con correos @totoninin.test y claves marcadas "Test/Prueba" (p. ej. admin@totoninin.test, user@totoninin.test). Ningún dato personal real se usó en ninguna prueba.

Reglas de trabajo seguidas

- Se auditó únicamente la aplicación propia del alumno (TotoNinin); no se atacó ningún servicio externo.
- Se usaron exclusivamente cuentas y datos de prueba, nunca información real.
- Las claves, hashes y cookies de sesión se ocultan en todas las capturas de evidencia de este reporte (se muestra su formato, no su valor).

- Cada cambio quedó registrado como commit independiente en el repositorio Git entregado (sección «Entrega y repositorio»).
- Se conservó una copia íntegra de la versión original (v1-original) como respaldo antes de corregir, tal como exige la metodología.

Aclaración de alcance: MASVS aplicado a una aplicación web

OWASP MASVS fue diseñado para aplicaciones móviles nativas o híbridas distribuidas como binario (APK/IPA). La aplicación entregada para esta práctica, TotoNinin, es una aplicación web clásica renderizada en servidor (PHP/MySQL) y no incluye ningún artefacto móvil compilado. Siguiendo el propio material de la unidad — cuya plantilla de verificación (diapositivas 50 y 51) ya incorpora criterios agnósticos de plataforma como "HTTP o TLS mal validado" y "Secretos incluidos en Git" — se aplicaron los grupos de MASVS cuyo equivalente tiene sentido en una arquitectura cliente-servidor web: STORAGE, CRYPTO, AUTH, NETWORK, CODE y PLATFORM (en su parte de control de acceso a componentes administrativos y validación de archivos). Los controles intrínsecamente ligados a un binario móvil compilado y distribuido en el dispositivo del usuario — RESILIENCE (anti-tampering, ingeniería inversa de un ejecutable local) y la parte de PLATFORM referida a permisos del sistema operativo, WebViews y enlaces profundos — se marcaron No aplica, justificado en el «Checklist MASVS», porque el código fuente de TotoNinin nunca se ejecuta en el dispositivo del atacante: se ejecuta en el servidor.

Controles seleccionados

Grupo MASVS	Resultado del alcance
MASVS-STORAGE	Evaluado
MASVS-CRYPTO	Evaluado
MASVS-AUTH	Evaluado
MASVS-NETWORK	No aplica (justificado) — ver «Checklist MASVS»

Grupo MASVS	Resultado del alcance
MASVS-PLATFORM	Evaluado parcialmente (control de acceso y validación de archivos); resto no aplica — ver «Checklist MASVS»
MASVS-CODE	Evaluado
MASVS-RESILIENCE	No aplica (justificado) — ver «Checklist MASVS»
MASVS-PRIVACY	Evaluado (cumple, con una observación menor no bloqueante)

Procedimiento de prueba

Se siguieron las ocho fases de la metodología de la unidad: inventariar, inspección estática, prueba dinámica, registrar, priorizar, corregir, reprobar y entregar.

Inventario

- **Datos almacenados:** tabla users (nombre, correo, hash de contraseña), events (título, descripción, fecha, foto, tipo), comments (texto libre asociado a un evento), event_type (catálogo de categorías).
- **Funciones sensibles:** inicio de sesión, registro público de usuarios, alta/edición/eliminación de usuarios desde el panel admin, alta/edición/eliminación de eventos y categorías, subida de fotografías de evento, publicación de comentarios, cierre de sesión.
- **Permisos/roles:** dos niveles implícitos en el código original — "usuario autenticado" y "administrador" (decidido comparando el correo contra un valor fijo), sin tabla de roles.
- **Superficie expuesta sin autenticación:** index.php, paginas/registro.php, paginas/login.php, paginas/informacion.php, paginas/preguntas.php.

Inspección estática

Revisión manual del código PHP completo (2,673 líneas en 37 archivos) buscando: concatenación de entrada de usuario en SQL, credenciales embebidas, algoritmo de hash de contraseñas, validación de archivos subidos, configuración de cookies de sesión, manejo de errores y presencia de .gitignore. Se usó comparación línea a línea (diff) entre v1-original y v2-corregida para verificar que cada corrección es mínima y no modifica comportamiento no relacionado con el hallazgo.

Prueba dinámica

Se levantó la aplicación con el servidor embebido de PHP (php -S) contra la base de datos de prueba y se ejecutó un conjunto de pruebas HTTP automatizado y reproducible (pruebas/run_tests.py, incluido en el repositorio entregado), que para cada versión del código:

- Reinicia la base de datos de prueba a un estado conocido antes de cada caso (aislamiento entre pruebas).
- Envía peticiones HTTP reales (con y sin sesión, con payloads maliciosos y con datos legítimos) usando un cliente propio con manejo de cookies.
- Registra la evidencia (código de estado, cabeceras relevantes, fragmentos de respuesta) enmascarando automáticamente cookies de sesión y hashes antes de guardarlos.
- Compara el resultado contra el criterio de corrección de cada hallazgo y lo marca SEGURO/CUMPLE o VULNERABLE/NO CUMPLE.

En total se ejecutaron 24 casos de prueba: 19 casos de seguridad (distribuidos en los 6 hallazgos) y 5 casos de regresión funcional (login legítimo, calendario, panel de administración, registro público y página de inicio), para verificar que las correcciones no rompieron ninguna función existente.

Registrar y priorizar

Cada hallazgo se documentó con una ficha (ID, grupo MASVS, ubicación, pasos de reproducción, evidencia, probabilidad, impacto y nivel de riesgo), siguiendo la escala didáctica Riesgo = Impacto × Probabilidad (1–3 cada uno) de la unidad: 1–3 Bajo, 4–6 Medio, 7–8 Alto, 9 Crítico.

Resultados y evidencias

Resumen comparativo de los 24 casos de prueba ejecutados contra ambas versiones del código:

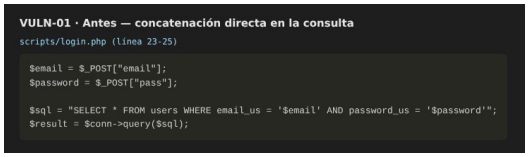
Versión	Casos seguros / cumplen	Casos vulnerables	Regresión funcional
v1-original	**8 / 24**	16 de 19 casos de seguridad	5 / 5 (ya funcionaba)
v2-corregida	**24 / 24**	0 de 19 casos de seguridad	5 / 5 (sin regresiones)

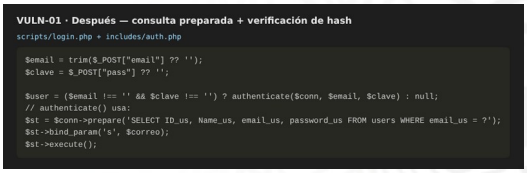
Evidencia por hallazgo (código y comportamiento, antes/después)

Las claves, hashes y cookies de sesión se muestran enmascarados (****) en toda la evidencia; solo se conserva su formato para poder verificar el control.

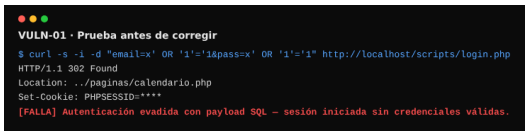
VULN-01 · Inyección SQL

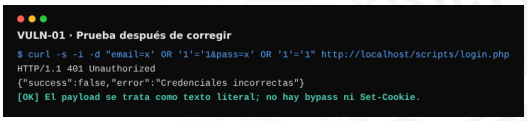
Código — antes (izquierda) y después (derecha):





Comportamiento observado — antes (izquierda) y después (derecha):





VULN-02 · Control de acceso roto

Código — antes (izquierda) y después (derecha):

VULN-02 · Antes — sin verificación de rol

scripts/eliminar_usuario.php (línea 1-16)

```php
<?php
session_start();

include "conexion.php";

if(empty($_GET['id'])){
    header("location: lista_usuarios.php");
} else {
    $id_usuario = $_GET['id'];
    ...
    $query_delete = mysqli_query($conection, "DELETE FROM users WHERE ID_us = $id_usuario");
}
```

VULN-02 · Después — control de acceso en el servidor

scripts/eliminar_usuario.php + includes/auth.php

```php
<?php
require_once __DIR__ . '/../includes/auth.php';
require_admin(); // VULN-02: exige sesión + rol administrador

function require_admin() {
    require_login();
    if (empty($_SESSION['es_admin'])) {
        http_response_code(403);
        exit('Acceso denegado');
    }
}
```

Comportamiento observado — antes (izquierda) y después (derecha):

VULN-02 · Prueba antes de corregir (sin sesión)

```
$ curl -s -i -X POST http://localhost/scripts/eliminar_usuario.php?id=3 -d id=3
HTTP/1.1 302 Found
Location: lista_usuarios.php
[FALLA] El usuario id=3 fue eliminado por un visitante sin sesión iniciada.
```

VULN-02 · Prueba después de corregir (sin sesión)

```
$ curl -s -i -X POST http://localhost/scripts/eliminar_usuario.php?id=3 -d id=3
HTTP/1.1 302 Found
Location: ../paginas/login.php
[OK] Redirigido al login; el registro permanece intacto en la base de datos.
```

VULN-03 · Hash de contraseñas débil

Código — antes (izquierda) y después (derecha):

VULN-03 · Antes — MD5 sin sal

scripts/registro_usuario.php (línea 17)

```php
$Name_us = $_POST['Name_us'];
$email_us = $_POST['email_us'];
$password_us = md5($_POST['password_us']); // sin sal, reversible por diccionario

$query_insert = mysqli_query($conection,
  "INSERT INTO users(Name_us,email_us,password_us) VALUES('$Name_us','$email_us','$password_us')");
```

VULN-03 · Después — bcrypt con sal + migración de cuentas antiguas

scripts/registro_usuario.php + includes/auth.php

```php
$password_us = password_hash($_POST['password_us'], PASSWORD_DEFAULT); // bcrypt con sal

$ins = $conection->prepare("INSERT INTO users(Name_us,email_us,password_us) VALUES(?,?,?)");
$ins->bind_param("sss", $Name_us, $email_us, $password_us);

// En el login, si la cuenta aún tiene un hash MD5 heredado, se valida una vez y se migra:
elseif (preg_match('/^[a-f0-9]{32}$/i', $hash) && hash_equals(strtolower($hash), md5($clave))) {
    $ok = true; $migrar = true;   // se reemplaza por bcrypt en la misma petición
}
```

Comportamiento observado — antes (izquierda) y después (derecha):

VULN-03 · Prueba antes de corregir

```
$ sqlite3 totoninin.db "SELECT password_us FROM users WHERE email_us='nuevo@totoninin.test'"
[hash MD5 de 32 caracteres — oculto]
[FALLA] Hash MD5 sin sal: crackeable con diccionario/rainbow table en segundos.
```

VULN-03 · Prueba después de corregir

```
$ sqlite3 totoninin.db "SELECT password_us FROM users WHERE email_us='nuevo@totoninin.test'"
[hash bcrypt $2y$10$... — oculto]
[OK] bcrypt con sal y factor de costo; no es reversible por diccionario.
```

VULN-04 · Subida de archivos sin validar

Código — antes (izquierda) y después (derecha):

VULN-04 · Antes — sin validar tipo ni contenido

scripts/registro_evento.php (línea 39-48)

```php
// Verificar si la foto es válida
if ($foto['error'] !== 0) {
    echo "Error: Foto inválida";
    exit;
}

// Mover la foto a una carpeta temporal
$rutaTemporal = $foto['tmp_name'];
$rutaDestino = '../fotos/' . $foto['name'];   // nombre y extensión del cliente, sin validar
move_uploaded_file($rutaTemporal, $rutaDestino);
```

VULN-04 · Después — valida MIME real y bloquea ejecución

includes/upload.php + fotos/.htaccess

```php
function guardar_foto_evento($f) {
    if (!is_uploaded_file($f['tmp_name']) || $f['size'] > 2*1024*1024) return null;
    $permitidos = ['image/jpeg'=>'jpg','image/png'=>'png','image/gif'=>'gif','image/webp'=>'webp'];
    $mime = (new finfo(FILEINFO_MIME_TYPE))->file($f['tmp_name']);     // MIME real, no el del navegador
    if (!isset($permitidos[$mime]) || @getimagesize($f['tmp_name']) === false) return null;
    $nombre = bin2hex(random_bytes(16)) . '.' . $permitidos[$mime];    // nombre aleatorio, no el del cliente
    move_uploaded_file($f['tmp_name'], __DIR__.'/../fotos/'.$nombre);
    return '../fotos/' . $nombre;
}
# fotos/.htaccess: php_flag engine off  (defensa en profundidad)
```

Comportamiento observado — antes (izquierda) y después (derecha):

VULN-04 · Prueba antes de corregir

```
$ curl -s -F "foto=@shell.php;type=image/jpeg" ... registro_evento.php
$ curl -s http://localhost/fotos/shell.php
RCE_42
[FALLA] El archivo .php subido se ejecutó en el servidor (ejecución remota de código).
```

VULN-04 · Prueba después de corregir

```
$ curl -s -i -F "foto=@shell.php;type=image/jpeg" ... registro_evento.php
HTTP/1.1 400 Bad Request
Error: archivo de imagen no válido
[OK] Rechazado por tipo MIME real; nunca se guarda ni se ejecuta.
```

VULN-05 · Secretos embebidos en el código

Código — antes (izquierda) y después (derecha):

VULN-05 · Antes — credenciales fijas en el repositorio

scripts/conexion.php

```php
<?php
$host = 'localhost';
$user = 'root';
$password = '';
$db = 'totoninin';

$conection = @mysqli_connect($host,$user,$password,$db);
```

VULN-05 · Después — credenciales fuera del código

scripts/conexion.php + includes/config.php

```php
<?php
require_once __DIR__ . '/../includes/db.php';
$conection = db();

// includes/config.php lee de variables de entorno o de
// includes/config.local.php (archivo excluido en .gitignore):
function cfg($clave, $defecto = null) {
    $valor = getenv($clave);
    ...
    return $local[$clave] ?? $defecto;
}
```

Comportamiento observado — antes (izquierda) y después (derecha):

VULN-05 · Prueba antes de corregir

```
$ grep -rn "new mysqli(\"localhost\", \"root\"" . | wc -l
14
[FALLA] 14 archivos con credenciales root/clave vacía embebidas en el código fuente.
```

VULN-05 · Prueba después de corregir

```
$ grep -rn "new mysqli(\"localhost\", \"root\"" . | wc -l
0
$ cat .gitignore | grep config.local
includes/config.local.php
[OK] Sin credenciales en el código; el archivo con secretos está excluido de Git.
```

VULN-06 · Gestión insegura de sesión

Código — antes (izquierda) y después (derecha):

VULN-06 · Antes — cookie sin protecciones, sin regenerar ID

scripts/login.php (linea 27-30)

```
if ($result->num_rows > 0) {
    session_start();                // cookie por defecto: sin HttpOnly ni SameSite
    $_SESSION["logged_in"] = true;
    $_SESSION["email"] = $email;    // mismo ID de sesión que antes del login (fijación)
    header("Location: ../paginas/calendario.php");
}
```

VULN-06 · Después — cookie segura + regeneración de ID

includes/session.php

```
session_set_cookie_params([
    'lifetime' => 0, 'path' => '/', 'secure' => $https,
    'httponly' => true, 'samesite' => 'Lax',
]);
session_start();
...
function login_user($correo, $nombre) {
    secure_session_start();
    session_regenerate_id(true);     // evita fijación de sesión
    $_SESSION['correo'] = $correo;
    $_SESSION['ultimo_acceso'] = time();   // expiración por inactividad
}
```

Comportamiento observado — antes (izquierda) y después (derecha):

VULN-06 · Prueba antes de corregir

```
$ curl -s -i -d "correo=user@totoninin.test&password=UserTest#2026" http://localhost/paginas/login.php
Set-Cookie: PHPSESSID=****; path=/
[FALLA] Cookie sin HttpOnly ni SameSite; además el ID de sesión no se regenera (fijación).
```

VULN-06 · Prueba después de corregir

```
$ curl -s -i -d "correo=user@totoninin.test&password=UserTest#2026" http://localhost/paginas/login.php
Set-Cookie: PHPSESSID=****; path=/; HttpOnly; SameSite=Lax
[OK] Cookie protegida; el ID de sesión se regenera en cada login (sin fijación).
```

Hallazgos y nivel de riesgo

Ficha obligatoria por hallazgo. Riesgo = Impacto × Probabilidad (escala 1–3 cada uno; 1–3 Bajo, 4–6 Medio, 7–8 Alto, 9 Crítico — escala didáctica de la unidad, no una escala oficial de OWASP MASVS).

VULN-01 — Inyección SQL (bypass de autenticación y exfiltración vía UNION)

Grupo MASVS	MASVS-CODE (primario) · MASVS-AUTH (secundario)
Ubicación	scripts/login.php (consulta de login); scripts/load_events.php, scripts/obtener_eventos.php (parámetro "mes"); scripts/lista_usuarios.php, lista_eventos.php, lista_categoria.php (parámetro "pagina"); scripts/registro_usuario.php, editar_usuario.php, registro_categoria.php, editar_categoria.php (correo/descripción)
Evidencia	Casos P-01a / P-01b — ver «Evidencia por hallazgo» y anexo de resultados

Reproducción:

1. Enviar POST a scripts/login.php con email = pass = x' OR '1'='1
2. El servidor concatena el valor directamente en el SQL y la condición siempre es verdadera: se inicia sesión sin conocer ninguna credencial.
3. Autenticado como un usuario cualquiera, enviar GET a scripts/load_events.php?mes=0 UNION SELECT ID_us,Name_us,password_us,... FROM users -- para leer los hashes de contraseña de todos los usuarios a través de una vista pensada solo para listar eventos del mes.

Probabilidad	Impacto	Riesgo
3 (alta)	3 (alto)	Crítico

Corrección aplicada:

Se sustituyeron todas las consultas que concatenaban entrada del usuario por sentencias preparadas (mysqli prepare/bind_param) y se forzó el tipo numérico (int)

en parámetros como "mes", "pagina" e "id" antes de usarlos en SQL. El login quedó centralizado en includes/auth.php::authenticate().

VULN-02 — Control de acceso roto en funciones administrativas

Grupo MASVS	MASVS-AUTH (primario) · MASVS-PLATFORM (secundario)
Ubicación	scripts/lista_usuarios.php, eliminar_usuario.php, editar_usuario.php, registro_usuario.php, lista_categoria.php, eliminar_categoria.php, registro_categoria.php, editar_categoria.php, lista_eventos.php, eliminar_confirmar_evento.php, registro_evento.php, editar_evento.php, paginas/admin.php
Evidencia	Casos P-02a / P-02b / P-02c / P-02d — ver «Evidencia por hallazgo» y anexo de resultados

Reproducción:

1. Sin iniciar sesión, solicitar GET scripts/lista_usuarios.php: la lista completa de usuarios (nombre y correo) se muestra igualmente.
2. Sin iniciar sesión, enviar POST a scripts/eliminar_usuario.php?id=3: el usuario se elimina de la base de datos sin pedir ninguna credencial.
3. Sin iniciar sesión, enviar POST a scripts/editar_usuario.php?id=2 con una contraseña nueva: la contraseña de ese usuario queda sobrescrita (toma de cuenta) y el atacante puede iniciar sesión con la clave que impuso.
4. Con sesión de un usuario normal (no administrador), acceder a scripts/lista_usuarios.php: el panel administrativo se muestra igual porque ningún script valida el rol.

Probabilidad	Impacto	Riesgo
3 (alta)	3 (alto)	**Crítico**

Corrección aplicada:

Se creó includes/auth.php con require_login() y require_admin(); los 14 scripts administrativos ahora exigen sesión y rol de administrador (es_admin, calculado en el servidor a partir de ADMIN_EMAIL) antes de ejecutar cualquier lectura o escritura.

VULN-03 — Hash de contraseñas débil (MD5 sin sal)

Grupo MASVS	MASVS-CRYPTO (primario) · MASVS-STORAGE (secundario)
Ubicación	scripts/registro.php, scripts/registro_usuario.php (alta de usuarios vía panel admin)
Evidencia	Casos P-03a / P-03b — ver «Evidencia por hallazgo» y anexo de resultados

Reproducción:

1. Registrar un usuario desde scripts/registro_usuario.php y consultar la columna password_us en la base de datos.
2. El valor es un hash MD5 de 32 caracteres hexadecimales, sin sal: reversible en segundos con un diccionario o una tabla arcoíris si la base de datos llegara a filtrarse.
3. Nota: el formulario público de registro (paginas/registro.php) sí usaba password_hash()/bcrypt desde el original; el problema estaba acotado al alta hecha por el administrador y a la clase legada Registro.

Probabilidad	Impacto	Riesgo
2 (media)	2 (medio)	**Medio**

Corrección aplicada:

Se reemplazó md5() por password_hash($clave, PASSWORD_DEFAULT) (bcrypt) en todos los puntos de alta/edición. Se añadió además una migración transparente: si una cuenta antigua todavía tiene un hash MD5, el primer login correcto lo valida una vez y lo reemplaza por bcrypt automáticamente.

VULN-04 — Subida de archivos sin validar (ejecución remota de código)

Grupo MASVS	MASVS-CODE (primario) · MASVS-PLATFORM (secundario)
Ubicación	scripts/registro_evento.php, scripts/editar_evento.php (campo "foto")
Evidencia	Casos P-04a / P-04b / P-04c — ver «Evidencia por hallazgo» y anexo de resultados

Reproducción:

1. Con una cuenta normal (registro público, sin privilegios de administrador), crear un evento adjuntando un archivo "shell.php" cuyo contenido es <?php echo 'RCE_'.(6*7); ?>.
2. La aplicación solo valida $foto['error'] y mueve el archivo a fotos/ conservando el nombre y la extensión que envía el navegador, sin comprobar el tipo de contenido real.
3. Visitar directamente fotos/shell.php: el servidor interpreta el archivo como PHP y ejecuta el código (salida "RCE_42"), lo que equivale a ejecución remota de código con los permisos del servidor web.

Probabilidad	Impacto	Riesgo
3 (alta)	3 (alto)	**Crítico**

Corrección aplicada:

Se añadió includes/upload.php::guardar_foto_evento(): valida que el archivo provenga de una subida real (is_uploaded_file), limita el tamaño, verifica el tipo MIME real con fileinfo (no el que declara el navegador) contra una lista blanca de imágenes, confirma con getimagesize() que el contenido es una imagen válida, y lo guarda con un nombre aleatorio y una extensión derivada del MIME (nunca la del cliente). Se agregó además fotos/.htaccess que desactiva la ejecución de PHP en esa carpeta como defensa en profundidad.

VULN-05 — Credenciales y secretos embebidos en el código fuente

Grupo MASVS	MASVS-STORAGE (primario) · MASVS-CODE (secundario)
Ubicación	scripts/conexion.php, bd_php/conexion.php, scripts/login.php, scripts/add_comment.php, scripts/evento_detalle.php, scripts/registro_evento.php, scripts/editar_evento.php, paginas/login.php, paginas/registro.php, index.php, scripts/load_events.php, scripts/obtener_eventos.php, scripts/eventos.php, scripts/registro.php, bd_php/usuarioBD.php
Evidencia	Casos P-05a / P-05b / P-05c — ver «Evidencia por hallazgo» y anexo de resultados

Reproducción:

1. Buscar en el código fuente la cadena new mysqli("localhost","root","",...): aparece repetida en 14 archivos, con el usuario root y la contraseña vacía escritos de forma literal.
2. bd_php/usuarioBD.php tenía además error_reporting(E_ALL) + ini_set("display_errors",1) y mostraba el mensaje de excepción de MySQL directamente al usuario (die("Error de conexión: " . $e->getMessage())), lo que expone detalles internos (rutas, nombres de tabla, versión de MySQL) a cualquier visitante.
3. El proyecto no incluía archivo .gitignore, por lo que un futuro archivo de configuración con credenciales reales se habría subido al repositorio sin control.

Probabilidad	Impacto	Riesgo
2 (media)	2 (medio)	Medio

Corrección aplicada:

Se centralizó la conexión en includes/db.php + includes/config.php, que leen host/usuario/clave/BD desde variables de entorno o desde includes/config.local.php (archivo que no se sube al repositorio). Los errores de conexión y de consulta ahora se registran con error_log() y al usuario solo se le muestra un mensaje genérico. Se

agregó .gitignore excluyendo config.local.php, *.env y los registros temporales de prueba.

VULN-06 — Gestión insegura de sesión (cookie sin protecciones, fijación de sesión, enumeración de usuarios y cierre de sesión incompleto)

Grupo MASVS	MASVS-AUTH (primario)
Ubicación	scripts/login.php, paginas/login.php, scripts/cerrar_sesion.php, includes/header.php, includes/nav.php
Evidencia	Casos P-06a / P-06b / P-06c / P-06d — ver «Evidencia por hallazgo» y anexo de resultados

Reproducción:

1. Al iniciar sesión, la cabecera Set-Cookie del PHPSESSID no incluye HttpOnly ni SameSite, y el identificador de sesión es el mismo que tenía el visitante antes de autenticarse (no se llama a session_regenerate_id()), lo que permite fijación de sesión: un atacante que induce a la víctima a usar un ID de sesión conocido (por ejemplo vía enlace) puede reutilizar ese mismo ID después de que la víctima inicie sesión.
2. El login devuelve mensajes distintos para "correo no existe" y "contraseña incorrecta", lo que permite enumerar qué correos están registrados probando uno por uno.
3. scripts/cerrar_sesion.php llama a session_destroy() pero nunca borra la cookie del navegador (no hay setcookie(...,-1)), por lo que el ID de sesión sigue viajando en cada petición aunque ya no tenga datos asociados.
4. El rol de administrador se decide comparando el correo contra un valor fijo dentro de includes/nav.php, repetido en cada archivo en vez de calcularse una sola vez en el servidor al autenticar.

Probabilidad	Impacto	Riesgo
2 (media)	2 (medio)	**Medio**

Corrección aplicada:

Se creó includes/session.php: cookie de sesión con HttpOnly, SameSite=Lax y Secure quién corresponda según HTTPS; expiración por inactividad (30 min); session_regenerate_id(true) en cada login (evita fijación); logout_user() que vacía $_SESSION, destruye la sesión en el servidor y expira la cookie en el navegador. El login ahora responde siempre "Credenciales incorrectas" sin distinguir el motivo, y el rol es_admin se calcula una única vez en el servidor al autenticar.

Correcciones implementadas

Resumen de los archivos nuevos y modificados entre v1-original y v2-corregida. El detalle línea por línea de cada cambio está disponible como diff en el repositorio entregado (carpeta del commit "Correcciones VULN-01..06").

Archivos nuevos (infraestructura de seguridad reutilizable)

Archivo	Propósito
includes/config.php	Lee configuración (host/usuario/clave de BD, correo de administrador) desde variables de entorno o desde includes/config.local.php, nunca desde literales en el código.
includes/ config.local.example.php	Plantilla de ejemplo para que cada entorno (desarrollo/producción) cree su propio includes/config.local.php, excluido de Git.
includes/db.php	Función db(): entrega una conexión mysqli ya configurada; ante un fallo registra el detalle en el log del servidor y muestra un mensaje genérico al usuario (VULN-05).
includes/session.php	secure_session_start() (cookie con HttpOnly/SameSite, expiración por inactividad), login_user() (regenera el ID de sesión) y logout_user() (destruye la sesión y expira la cookie) — VULN-06.
includes/auth.php	require_login(), require_admin() (VULN-02) y authenticate() con consulta preparada, password_verify() y migración automática de hashes MD5 heredados a bcrypt (VULN-01 / VULN-03).
includes/upload.php	guardar_foto_evento(): valida tipo MIME real, tamaño y estructura de imagen, y genera un nombre de archivo aleatorio (VULN-04).
fotos/.htaccess	Desactiva la ejecución de PHP dentro de la carpeta de imágenes subidas por usuarios, como defensa en profundidad adicional a VULN-04.
.gitignore	Excluye includes/config.local.php, *.env y archivos temporales de prueba del control de versiones (VULN-05).

Archivos modificados

Archivo	Corrección aplicada
scripts/login.php, paginas/login.php, paginas/login.js	VULN-01 (consulta preparada), VULN-06 (sesión segura, mensaje genérico, textContent en vez de innerHTML)
scripts/conexion.php, bd_php/conexion.php, index.php, scripts/{load_events,obtener_eventos,evento_detalle,add_comment,eventos,registro}.php, paginas/registro.php, bd_php/usuarioBD.php	VULN-05 (credenciales fuera del código, errores sin detalle al usuario)
scripts/ {registro_usuario,editar_usuario, registro_categoria,editar_categoria,eliminar_usuario,eliminar_categoria,eliminar_confirmar_evento,lista_usuarios,lista_categoria,lista_eventos,buscar_usuario}.php, paginas/admin.php	VULN-02 (require_admin), VULN-01 (consultas preparadas, parámetros numéricos forzados a int)
scripts/registro_evento.php, scripts/editar_evento.php	VULN-04 (validación real del archivo subido vía includes/upload.php)
scripts/cerrar_sesion.php, includes/header.php, includes/nav.php	VULN-06 (logout_user(), rol es_admin calculado en el servidor)

Repetición de pruebas y conclusiones

Reprueba

Se ejecutó el mismo conjunto de 24 pruebas automatizadas contra v2-corregida: 24/24 casos resultaron SEGURO/CUMPLE, incluyendo los 5 casos de regresión funcional (login legítimo, calendario de eventos, panel de administración, registro público y página de inicio), lo que confirma que las correcciones eliminan los 6 hallazgos sin romper ninguna función existente de la aplicación.

Riesgos pendientes y recomendaciones adicionales

- MASVS-NETWORK quedó fuera del alcance evaluable porque el proyecto no incluye la configuración del servidor web de producción; se recomienda forzar HTTPS y agregar la cabecera Strict-Transport-Security en ese servidor.
- No se implementó límite de intentos de inicio de sesión (rate limiting); se recomienda agregarlo para mitigar fuerza bruta residual, incluso con contraseñas ya en bcrypt.
- No se agregaron cabeceras de seguridad adicionales como Content-Security-Policy; se dejaron únicamente X-Content-Type-Options, X-Frame-Options y Referrer-Policy por ser cambios mínimos y no romper recursos externos ya usados por la app (fuentes e íconos de terceros).
- La observación menor de MASVS-PRIVACY (falta de aviso de privacidad y opción de autoeliminación de cuenta) se deja como mejora futura, fuera de las 6 correcciones obligatorias de esta auditoría.
- En un despliegue real, la base de datos debe usar un usuario de MySQL con privilegios mínimos y contraseña fuerte (nunca root sin clave); includes/config.local.example.php documenta ese cambio.

Conclusiones

La revisión identificó 6 fallas verificables (uno más del mínimo de 5 solicitado), repartidas en 3 de riesgo Crítico (inyección SQL, control de acceso roto y subida de archivos sin validar) y 3 de riesgo Medio (hash débil de contraseñas, secretos embebidos en el código y gestión insegura de sesión). Las 6 se corrigieron y se verificaron con pruebas dinámicas reproducibles antes y después del cambio, además de 5 pruebas de regresión que confirman que la funcionalidad original de la aplicación (inicio de sesión, calendario, panel de administración, registro público) se conserva intacta. Los controles MASVS-RESILIENCE y la parte de MASVS-PLATFORM ligada a permisos de sistema operativo se marcaron No aplica porque TotoNinin es una aplicación web sin empaquetado móvil compilado, justificación que se documenta en el checklist siguiente.

Checklist MASVS

Área	Qué se revisó	Estado	Evidencia / justificación
STORAGE	Credenciales embebidas, gestión de secretos, errores detallados expuestos, exclusión de secretos de Git.	NC	VULN-05 (P-05a/b/c) — corregido, ver reprueba 24/24.
CRYPTO	Algoritmo de hash para contraseñas, generación de nombres de archivo/identificadores.	NC	VULN-03 (P-03a/b) — MD5→bcrypt, corregido.
AUTH	Inyección SQL en login, control de acceso a funciones administrativas, cookie de sesión, fijación de sesión, enumeración de usuarios, cierre de sesión.	NC	VULN-01, VULN-02, VULN-06 — corregidos; ver fichas y reprueba.

Área	Qué se revisó	Estado	Evidencia / justificación
NETWORK	Transporte cliente-servidor (HTTP/TLS).	N/A	El proyecto entregado no incluye configuración de servidor web (vhost de Apache/Nginx) ni certificado para un despliegue de producción; no se encontraron endpoints propios codificados con http:// (solo recursos externos de terceros como fuentes e íconos). Queda fuera del alcance del código fuente evaluado; se recomienda forzar HTTPS y la cabecera Strict-Transport-Security en el servidor de producción.
PLATFORM	Validación de archivos subidos al servidor (equivalente web a permisos/IPC); control de acceso a componentes administrativos.	NC	VULN-04 (subida de archivos) y VULN-02 (acceso a paneles) — corregidos. No aplican WebViews, enlaces profundos ni permisos del sistema operativo porque la aplicación entregada es una aplicación web PHP renderizada en servidor, sin empaquetado móvil (APK/IPA) en el proyecto evaluado.
CODE	Validación de entradas, manejo de errores, modo de depuración.	NC	VULN-01 (inyección SQL) y VULN-05 (display_errors) — corregidos.
RESILIENCE	Manipulación, ingeniería inversa, controles de entorno de ejecución.	N/A	No aplica: la aplicación no se distribuye como binario compilado (APK/IPA) sino como código PHP interpretado en el servidor; no existe un artefacto cliente que un atacante pueda descompilar o manipular localmente. El código fuente del servidor nunca llega al dispositivo del usuario, que es la premisa de los controles de resistencia de MASVS.
PRIVACY	Minimización de datos, consentimiento, rastreadores, retención.	C	Los datos solicitados (nombre, correo, contraseña, comentarios) son los mínimos necesarios para la función de la app. No se detectaron scripts de rastreo/analítica de terceros en el código. Observación menor (no bloqueante): no existe un aviso de privacidad ni opción de eliminar la cuenta propia; se deja como recomendación para una iteración futura, fuera de las 6 correcciones de esta auditoría.

C = cumple · NC = no cumplía (corregido, reprobado y hoy cumple) · N/A = no aplica (justificado arriba)

Entrega y repositorio

El repositorio entregado contiene:

- v1-original/ — copia íntegra de la aplicación tal como fue recibida (snapshot del ZIP), conservada como respaldo antes de corregir.
- v2-corregida/ — la misma aplicación con las 6 correcciones aplicadas.
- pruebas/ — script de pruebas automatizado (run_tests.py), capa de compatibilidad de base de datos para el entorno de auditoría (mysqli_shim.php) y los resultados crudos de cada ejecución (resultados.txt / resultados.json) para ambas versiones.
- evidencias/ — las capturas de código y de comportamiento (antes/después) usadas en este reporte.
- reporte/ — este documento.

Historial de Git: el repositorio se inicializó con un commit "v1-original: snapshot de la aplicación entregada" y un segundo commit "Correcciones VULN-01..06 (reprueba 24/24)" que introduce exclusivamente los cambios descritos en «Correcciones implementadas», de modo que el historial permite comparar antes/después con git diff.

Pendiente de indicar por el alumno: URL del repositorio remoto (GitHub/GitLab) donde se debe subir — este documento y el código quedan listos para el push en cuanto se confirme el destino.